

HoloAssist

Technical Documentation

XR Teleoperation & Collaboration Framework — Meta Quest 3 + UR3e + OnRobot RG2

Sprint 4 | Version 1.0 | 2026-05-13 | github.com/John-A-Chen/HoloAssist

Team Member	Subsystem
John Chen	Perception
Oliver Lau	Autonomous Control
Nicholas Sabatta	Teleoperation and Interaction
Sebastian Baudille	XR Scene and Visualisation

Table of Contents

Right-click in Word and select 'Update Field' to populate page numbers.

1. Project Overview

HoloAssist is a real-time Extended Reality (XR) teleoperation and collaboration framework integrating Meta Quest 2/3 headsets with a collaborative UR3e robot arm and OnRobot RG2 gripper. The system enables intuitive human-robot collaboration by combining XR visual overlays (live robot data, bin sorting status, TF axis visualisation) with real-time robot control via Jacobian-based velocity commands, MoveIt 2 trajectory planning, and mixed-reality operator feedback.

1.1 Operating Modes

Mode	Description	Primary Interface
Teleoperation	Operator wears Quest 3 and directly controls the UR3e via RMRC, Direct Joint, or Hand Guide	Meta Quest 3 controllers + RobotController.cs
Autonomous	Robot autonomously sorts detected cubes into bins using MoveIt 2; no operator input	ros2 service call /holoassist/pick_cube_to_bin ...

1.2 Key Features

Feature	Description
AprilTag Perception	RealSense D435i + apriltag_ros detects workspace board and 4 cubes; metric poses in workspace_frame
MoveIt 2 Autonomous Sorting	Collision-free pick-and-place: Cartesian approach + OMPL fallback; configurable bin destinations
XR Teleoperation (3 modes)	RMRC (DLS Jacobian), Direct Joint, Hand Guide; 50 Hz joint velocity via ros_tcp_endpoint
XR Heads-Up Display	RobotDataPanel (joints, EE pose, mode, gripper %); BinStatusPanel (per-bin count); RadialMenu (13 toggles)
Mixed Reality / VR toggle	PassthroughToggle switches MR (passthrough) and full VR
React Web Dashboard	Remote monitoring without a headset; same rosbridge data source
Session Logging + E-stop	SessionLogger records mode durations; PyQt5 dashboard E-stop: burst zeros + controller switch

1.3 Subsystem List

#	Subsystem	Owner	Package(s)
1	Perception	John Chen	ros2_ws/src/holo_assist_depth_tracker/
2	Autonomous Sorting	Oliver Lau	ros2_ws/src/moveit_robot_control/ + moveit_robot_control_msg
3	Teleoperation & Interaction	Nicholas Sabatta (Nic)	Unity/Assets/Scripts/ + dashboard/ + holoassist_unity_bridge
4	XR Scene & Visualisation	Sebastian Baudille	Unity/Assets/ (Scenes/UI)

1.4 Coordinate Frame Reference

Frame	Origin / Purpose	Publisher	Consumers
base_link	UR3e robot base (MoveIt planning frame)	ur_onrobot_control driver	Everything — canonical reference
workspace_frame	Workspace board origin (AprilTag corner 0)	holoassist_workspace_board_node	Perception, Sorting adapter, RViz
camera_color_optical_frame	RealSense D435i optical centre	RealSense driver	Perception, calibration
apriltag_cube_1 through _4	Detected cube centres from face-tag consensus	holoassist_cube_pose_node	Sorting adapter, RViz
tool0	UR3e end-effector frame	robot_state_publisher	Teleop Jacobian, RViz
unity_cube	Interactive Unity cube pose (placeholder)	ROSObjectPublisher (XR subsystem)	RViz

1.5 System Architecture Diagram

2. Dependencies

2.1 Hardware — Bill of Materials

Component	Model / Spec	Qty	Notes
Laptop (ROS + Unity host)	Ubuntu 22.04 LTS native; 16+ GB RAM; modern GPU	1	Native Ubuntu required (not WSL) for UR driver
XR Headset	Meta Quest 3 (primary); Quest 2 compatible	1	Developer Mode + USB debugging enabled
Robot arm	Universal Robots UR3e	1	IP: 192.168.0.194; External Control URCap required on teach pendant
Gripper	OnRobot RG2	1	Modbus interface via robot controller; 0-85 mm jaw range
RGB-D Camera	Intel RealSense D435i	1	USB 3.0 (blue port); ceiling-mounted above workspace
Robot router	2.4 / 5 GHz dual-band, DHCP	1	Provides 192.168.0.x subnet for all devices
Workspace board	700 mm x 500 mm with AprilTags at corners (IDs 0-3)	1	Tag edge exactly 32 mm — must match config/apriltag_all.yaml
Cubes	Physical cubes with 6-face AprilTags (IDs 10-33 across 4 cubes)	4	Tag edge exactly 32 mm; cube edge 40 mm

2.2 Computing Requirements

Requirement	Spec
OS	Ubuntu 22.04 LTS native (not WSL)
RAM	16+ GB
GPU	Modern GPU recommended for Unity XR rendering
Network	Wi-Fi on 192.168.0.x subnet (robot router); Ethernet at 192.168.0.1xx to UR3e
Headset	Meta Quest 3, Developer Mode, same Wi-Fi subnet as laptop

2.3 Software Dependencies

Component	Version	How to Install
ROS 2	Humble Hawksbill	curl + apt — see Section 3.2
Unity Editor	6.3 LTS (6000.3.9f1)	Unity Hub - Installs; add Android Build Support module
Meta XR All-in-One SDK	v85.0.0	Unity Package Manager (inside project)
ur_onrobot_control	latest main-ros2	git clone github.com/tonydle/OnRobot_ROS2_Driver into ros2_ws/src/
ROS-TCP-Connector	v0.7.0 (local)	Clone into ROS-TCP-Connector/ at repo root; manifest.json file:// reference
URDF-Importer	local	Clone into URDF-Importer/ at repo root; manifest.json file:// reference
apriltag_ros	humble	sudo apt install ros-humble-apriltag-ros
RealSense SDK	humble	sudo apt install ros-humble-realsense2-camera
MovelT 2	humble	sudo apt install ros-humble-moveit
Easy-Handeye2	local clone	Clone into ros2_ws/src/easy_handeye2/; colcon build --packages-select easy_handeye2
Node.js	18+	node --version to verify; install from nodejs.org
React + Vite	via npm	cd holoassist-dashboard && npm install

3. Installation

3.1 Hardware Setup

1. Mount the RealSense D435i above the workspace on a rigid stand. Connect to laptop via USB 3.0 (blue port).
2. Place the workspace board (700 mm x 500 mm) in the robot workspace. Ensure all four corner AprilTags (IDs 0-3) are unobstructed.
3. Place the four cubes with AprilTag faces visible to the camera (at least one face per cube).
4. Connect the UR3e to the laptop via Ethernet: laptop Ethernet -> robot router; set laptop Ethernet IP to 192.168.0.100. Robot IP: 192.168.0.194.
5. Connect laptop Wi-Fi to the robot router (192.168.0.x subnet). Confirm Quest 3 is on the same router.
6. Power on the UR3e. On the teach pendant: load and run the External Control program (host IP: 192.168.0.100).

3.2 Software Setup

1. Install ROS 2 Humble

```
curl -fsSL https://repo.ros2.org/ros.key | sudo apt-key add -  
echo "deb [arch=amd64] http://repo.ros2.org/ubuntu $(lsb_release -cs) main" | sudo tee  
/etc/apt/sources.list.d/ros2.list  
sudo apt update && sudo apt install -y ros-humble-desktop  
source /opt/ros/humble/setup.bash && rosdep init && rosdep update
```

2. Clone the repository and external packages

```
git clone https://github.com/John-A-Chen/HoloAssist.git HoloAssist_Main && cd  
HoloAssist_Main  
git clone git@github.com:Tony-Leu/UR_OnRobot_ROS2.git ros2_ws/src/ur_onrobot_control  
git clone https://github.com/Unity-Technologies/ROS-TCP-Connector.git  
git clone https://github.com/Unity-Technologies/URDF-Importer.git
```

3. Install ROS packages and dependencies

```
sudo apt install -y ros-humble-apriltag-ros ros-humble-realsense2-camera ros-humble-  
moveit  
cd ros2_ws && rosdep install --from-paths src --ignore-src -r -y
```

4. Build the ROS workspace

```
colcon build --symlink-install && source install/setup.bash
```

5. Install the web dashboard

```
cd ../holoassist-dashboard && npm install
```

6. Verify installation

```
ros2 pkg list | grep holo_assist  
# Expected: holo_assist_depth_tracker    holo_assist_depth_tracker_sim
```

4. Running the System

4.1 Quick Start — Virtual Test (No Robot)

```
# Terminal 1 – full ROS stack (fake hardware)
cd HoloAssist_Main && ./launch.sh
# Terminal 2 – E-stop dashboard (optional)
./dashboard.sh --fullscreen
# Terminal 3 – web dashboard (optional)
cd holoassist-dashboard && npm run dev    # open http://localhost:5173
# Unity: File -> Play | ROS Settings: IP = 127.0.0.1, Port = 10000
```

4.2 Real Hardware Launch

```
cd HoloAssist_Main && ./launch.sh --robot-ip 192.168.0.194
# On teach pendant: Load and run External Control (host IP 192.168.0.100)
# Unity ROS Settings: IP = <laptop WiFi IP e.g. 192.168.0.102>, Port = 10000
```

4.3 Expected Behaviour After Launch

1. `ros_tcp_endpoint` starts on port 10000; beacon.py UDP multicast allows Quest auto-discovery.
2. Unity connects; two floating panels appear in the Quest headset (joint data left, bin status right).
3. Panel status shows 'Connected'; joint angles update live.
4. Perception: `workspace_mode` reports 'locked' once all four board corner tags are visible.
5. Cube poses appear on `/holoassist/perception/april_cube_N_pose` at ~10 Hz when cubes in view.
6. Pressing Y on the left controller opens the 13-button radial menu.

[Insert screenshot of expected headset view and terminal output here]

4.4 Deploy to Quest 3

```
cd Unity/My\ project
# File -> Build Settings -> Android -> Build and Run
# Manual fallback: adb install -r Builds/HoloAssist.apk
# On Quest: App Library -> Unknown Sources -> HoloAssist
```


5. Subsystems

Template

5.1 Perception

Owner: John Chen | **Tech stack:** Python 3.10 / ROS 2 Humble / apriltag_ros / Intel RealSense SDK 2.x

Package(s): ros2_ws/src/holo_assist_depth_tracker/ ros2_ws/src/holo_assist_depth_tracker_sim/

5.1.1 Walkthrough Script (approx. 5 minutes)

Tick each step as you cover it. Sections below are in the same order — scroll down as you present.

Walkthrough Script — follow this with your coach

Step 1. Orient your coach: 'Perception detects the four cubes and workspace board using AprilTag markers on a RealSense D435i camera, and publishes metric poses so the Sorter can pick them and the XR scene can display them.'

Step 2. Show Purpose & Design Rationale — explain why AprilTags were chosen over depth-based clustering.

Step 3. Show How It Works — two-stage pipeline: workspace board lock (SVD/Kabsch), then per-cube multi-face consensus.

Step 4. Open holoassist_workspace_board_node.py and holoassist_cube_pose_node.py — point to the SVD/Kabsch call and the face-consensus averaging loop.

Step 5. Show Inputs/Outputs — emphasise cube poses are in workspace_frame, not base_link.

Step 6. Run the simulation twin: ros2 launch holo_assist_depth_tracker sim_holoassist_trolley.launch.py gui:=true — narrate topic output.

Step 7. Open Parameter Reference — explain apriltag_size: mismatch causes proportional metric drift.

Step 8. Walk through one Troubleshooting row with reasoning.

Step 9. Close with Limitations — one-time workspace lock, exact tag size requirement.

Step 10. Invite questions.

5.1.2 Purpose & Design Rationale

Purpose:

Detects the four sortable cubes and the shared workspace board in real time using AprilTag fiducial markers on a ceiling-mounted Intel RealSense D435i. Publishes metric 3-D poses for each cube in a stable workspace_frame so Autonomous Sorting can plan pick-and-place motions and XR Scene can overlay status information at the correct physical locations.

Role in the integrated system:

Subscribes to raw colour and depth images from the RealSense driver. Publishes per-cube poses consumed by Autonomous Sorting (via the pose-handoff adapter) and colour-coded RViz markers for debugging.

Key design decisions and why:

- AprilTag markers instead of depth-based clustering — tags give metric 6-DOF pose in a single frame with no hyperparameter tuning; clustering requires careful RANSAC+DBSCAN thresholds and breaks down for similarly-shaped objects.
- Two-stage pipeline (board lock first, then cube poses) — workspace_frame from the four corner tags (IDs 0-3) provides a stable coordinate origin that survives minor camera movement without recalibrating the full TF chain.
- Multi-face consensus per cube (6 face tags, IDs 10-33) — averaging visible face measurements reduces pose variance from partial occlusion; a single visible face is sufficient but noisier.
- Simulation twin (depth_tracker_sim) publishes on the same topics as the real node — downstream subsystems are unaffected and can be developed without the physical camera.

5.1.3 How It Works — Internal Logic

Point to the implementing code for each step during the walkthrough.

Processing pipeline:

1. RealSense driver publishes /camera/camera/color/image_raw and /camera/camera/depth/image_rect_raw at 30 Hz.
2. apriltag_ros detects all visible AprilTag IDs and publishes raw detections on /detections_all.
3. holoassist_workspace_board_node.py reads tags 0-3 (board corners), computes workspace_frame via SVD/Kabsch alignment, and broadcasts a static TF. Once locked, the frame does not update until /holoassist/perception/realign_workspace is called.
4. holoassist_cube_pose_node.py reads tags 10-33 (6 face tags per cube, 4 cubes). For each visible face tag, it computes the cube centre by applying a 20 mm face-to-centre offset. Visible face results are averaged (consensus). Zero faces = 'not_detected'.
5. Cube centres published as geometry_msgs/PoseStamped in workspace_frame on /holoassist/perception/april_cube_N_pose at ~10 Hz when detected.
6. Per-cube status published on /holoassist/perception/april_cube_N_status.
7. Colour-coded RViz markers published on /holoassist/perception/april_cube_N_marker (1=red, 2=green, 3=blue, 4=orange).

TIP Open the source file and point to the lines implementing each step above.

Data flow:

[Insert: RealSense colour/depth -> apriltag_ros /detections_all -> workspace_board_node (SVD) -> workspace_frame TF; detections -> cube_pose_node (face consensus) -> /holoassist/perception/april_cube_N_pose in workspace_frame -> Sorting adapter + RViz]

5.1.4 Key Files & Code Structure

File / Node	Path	Role
holoassist_workspace_board_node.py	ros2_ws/src/holo_assist_depth_tracker/holo_assist_depth_tracker/	Solves a workspace from corner points (0-3) (SVD/Kalman)
holoassist_cube_pose_node.py	ros2_ws/src/holo_assist_depth_tracker/holo_assist_depth_tracker/	Multi-face consensus per cube (33)
holo_assist_depth_tracker_node.py	ros2_ws/src/holo_assist_depth_tracker/holo_assist_depth_tracker/	Depth in and depth overlay
holoassist_4tag_board_4cube.launch.py	ros2_ws/src/holo_assist_depth_tracker/launch/	Main hardware launch: RealSense, apriltag, both nodes
sim_holoassist_trolley.launch.py	ros2_ws/src/holo_assist_depth_tracker/launch/	Simulation of synthetic trolley published in Gazebo
config/apriltag_all.yaml	ros2_ws/src/holo_assist_depth_tracker/config/	All 34 tag physical dimensions (0.032 m)
config/workspace.yaml	ros2_ws/src/holo_assist_depth_tracker/config/	Board geometry (0.7 m x 0.7 m) tag size
config/cubes.yaml	ros2_ws/src/holo_assist_depth_tracker/config/	Cube ID and face centre of mass mapping

5.1.5 Inputs & Outputs

Inputs:

Input	Source	Data Type / Format	Notes
/camera/camera/color/image_raw	RealSense driver	sensor_msgs/Image	30 Hz — consumed by apriltag_ros
/camera/camera/depth/image_rect_raw	RealSense driver	sensor_msgs/Image	30 Hz — depth integration
/camera/camera/color/camera_info	RealSense driver	sensor_msgs/CameraInfo	Camera intrinsics for tag pose computation
/detections_all	apriltag_ros	apriltag_msgs/AprilTagDetectionArray	All visible tag poses consumed by both board and cube nodes

Outputs:

Output	Destination	Data Type / Format	Notes
/holoassist/perception/april_cube_N_pose (N=1-4)	Autonomous Sorting adapter	geometry_msgs/PoseStamped	Cube centre in workspace_frame, ~10 Hz
/holoassist/perception/april_cube_N_status (N=1-4)	Diagnostics / XR Scene	std_msgs/String	'detected' or 'not_detected' per cube
/holoassist/perception/april_cube_N_marker (N=1-4)	RViz	visualization_msgs/Marker	Colour-coded markers
/holoassist/perception/workspace_mode	All subsystems	std_msgs/String	'locked' / 'searching'
TF: workspace_frame	All subsystems	tf2_msgs/TFMessage	Stable board-origin frame (static once locked)

5.1.6 ROS Interfaces

Published topics:

Topic	Msg Type	Rate	Description
/holoassist/perception/april_cube_N_pose	geometry_msgs/PoseStamped	~10 Hz	Cube centre in workspace_frame
/holoassist/perception/april_cube_N_status	std_msgs/String	~10 Hz	Detection state per cube

Topic	Msg Type	Rate	Description
/holoassist/perception/april_cube_N_marker	visualization_msgs/Marker	~10 Hz	RViz colour-coded marker
/holoassist/perception/workspace_mode	std_msgs/String	On change	'locked' or 'searching'

Subscribed topics:

Topic	Msg Type	Description / How Used
/detections_all	apriltag_msgs/AprilTagDetectionArray	Raw AprilTag detections — consumed by both nodes
/camera/camera/color/camera_info	sensor_msgs/CameraInfo	Camera intrinsics for metric pose

Services / Actions:

Name	Type	Request -> Response	Description
/holoassist/perception/realign_workspace	std_srvs/Trigger	[empty] -> success: bool	Re-solves workspace_frame if the board moves

5.1.7 Running & Testing Independently

Prerequisites:

- RealSense D435i plugged into USB 3.0 (blue port).
- AprilTags printed at exactly 32 mm edge size on board corners and cube faces.
- `source /opt/ros/humble/setup.bash && source ros2_ws/install/setup.bash`

Launch commands:

```
# Hardware
source /opt/ros/humble/setup.bash && source ros2_ws/install/setup.bash
ros2 launch holo_assist_depth_tracker holoassist_4tag_board_4cube.launch.py
# Simulation (no camera needed)
ros2 launch holo_assist_depth_tracker sim_holoassist_trolley.launch.py gui:=true
```

Test input:

```
ros2 topic echo /holoassist/perception/workspace_mode --once
ros2 topic echo /holoassist/perception/april_cube_1_pose --once
```

Verify output:

```
ros2 topic hz /holoassist/perception/april_cube_1_pose # expect ~10 Hz when cube 1 visible
```

EXPECTED workspace_mode reports 'locked' within a few seconds. Detected cube topics publish at ~10 Hz. In RViz, colour-coded markers appear at the correct physical positions relative to the board.

[Insert screenshot of expected terminal output / headset view here]

5.1.8 Parameter Reference

Coaches often ask 'what happens if I change X?'. The two right columns are the HD differentiator.

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
apriltag_size	config/apriltag_all.yaml	0.032 m	Must match physical print	N/A — physical property	Any mismatch causes proportional metric drift: 10% size error = 10% pose error
workspace board dims (0.7 x 0.5 m)	config/workspace.yaml	0.700 / 0.500 m	Must match physical board	N/A — physical property	Wrong dims skew the SVD alignment; board corner tags won't align in workspace_frame
cube_edge_size	config/cubes.yaml	0.040 m	Must match physical cube	N/A — physical property	Wrong size offsets cube-centre estimate by half the error (face-to-centre = cube_edge / 2)
face_offset	config/cubes.yaml	0.020 m	cube_edge_size / 2	N/A — derived	Incorrect offset shifts all cube-centre estimates; reduces grasp accuracy
[Add apriltag_ros detector params...]	config/apriltag_all.yaml				

TIP Prepare to demonstrate one live parameter change — edit a value, show the behaviour difference.

5.1.9 Live Configuration Changes

Scenario — "Workspace moved — re-lock workspace_frame":

1. Call: `ros2 service call /holoassist/perception/realign_workspace std_srvs/Trigger`
2. Reasoning: the board node re-reads all four corner tag detections and re-runs SVD/Kabsch; `workspace_frame` shifts to match the board's new physical position.
3. Verify: `ros2 topic echo /holoassist/perception/workspace_mode` briefly reports 'searching' then 'locked'.

Scenario — "Switch to simulation (no camera / cubes needed)":

1. Stop the hardware launch if running.
2. `ros2 launch holo_assist_depth_tracker sim_holoassist_trolley.launch.py gui:=true`
3. Reasoning: the sim twin publishes synthetic poses on the same topics; Sorting and XR Scene are unaffected.

5.1.10 Troubleshooting (with Reasoning)

The 'Why the Fix Works' column separates HD from Credit.

Symptom	Likely Cause & Why	How to Diagnose	Fix	Why the Fix Works
<code>workspace_mode</code> stays 'searching'	Board corner tags 0-3 not all visible — SVD requires all four	<code>ros2 topic echo /detections_all</code> — check which IDs appear	Move camera until all four board corners are in frame	SVD/Kabsch alignment needs four non-collinear anchor points; fewer than four leaves the system under-determined
Cube pose drifts or jumps	<code>apriltag_size</code> in config does not match the physical printed size	Measure tag edge with calipers; compare to <code>config/apriltag_all.yaml</code>	Update <code>apriltag_size</code> to the measured value and re-launch	Tag pose is computed using declared size as metric scale; any error is a direct proportional offset in X/Y/Z
Cube reported 'not_detected' when visible	Visible face tag ID not in the expected range for that cube	<code>ros2 topic echo /detections_all</code> and check IDs	Confirm face labels on physical cube match <code>config/cubes.yaml</code> ID groups	Each cube's ID group must exactly match printed tags; a mislabelled face never contributes to consensus
No output after hardware launch	RealSense not found on USB	<code>ros2 run realsense2_camera realsense2_camera_node</code> — look for camera serial	Replug into USB 3.0 (blue); check udev rules for RealSense	RealSense needs USB 3.0 bandwidth for simultaneous colour+depth; USB 2.0

Symptom	Likely Cause & Why	How to Diagnose	Fix	Why the Fix Works
				silently prevents driver start
[Add more as discovered...]				

5.1.11 Limitations & Assumptions

Assumptions:

- AprilTags printed at exactly 32 mm edge — any deviation causes proportional metric drift.
- Camera rigidly mounted with a calibrated static TF to base_link (from easy_handeye2; see CALIBRATION.md).
- All four board corner tags (IDs 0-3) visible simultaneously during initialisation.
- Board does not move after workspace_frame is locked (or realign_workspace called after each move).

Known limitations:

- Workspace frame solved once and locked — moving board or camera requires manual realign_workspace call.
- Simulation twin uses pre-placed deterministic cube positions — does not simulate camera noise or occlusion.
- Partially occluded cubes (zero visible face tags) reported as 'not_detected', not estimated.
- No object classification — tracks cubes by AprilTag ID only.

Future work:

- Continuous workspace_frame updates with outlier rejection for robustness to occasional tag dropouts.
- Depth-fused pose refinement to improve metric accuracy beyond tag pose alone.
- Support more than 4 cubes by extending ID groups in config/cubes.yaml.

5.1.12 Anticipated Coach Questions

Pre-prepare answers so you respond confidently.

Q1: Why AprilTags instead of point-cloud clustering?

A1: AprilTags give a metric 6-DOF pose from a single image frame with no training data and no hyperparameter tuning. Point-cloud clustering requires careful threshold tuning per object shape and breaks down with similarly-sized adjacent objects. For known objects with printed tags, fiducial detection is faster, more reliable, and easier to debug.

Q2: What happens if one cube face tag is damaged?

A2: The consensus averages over all visible face tags. With 5 of 6 faces intact the estimate is unaffected. If only 1 face is visible the pose is noisier but still published. If 0 faces are visible the cube is reported 'not_detected' until a face comes into view.

Q3: How is workspace_frame related to base_link?

A3: workspace_frame is the board-corner origin published by the board node. The pose-handoff adapter looks up the TF from workspace_frame to base_link to transform cube poses into the MoveIt planning frame. The calibration transform is provided by easy_handeye2 (see CALIBRATION.md).

Q4: What is the latency from cube movement to published pose?

A4: apriltag_ros runs at the camera frame rate (30 Hz). The cube pose node processes each detection batch and publishes at ~10 Hz. End-to-end latency from physical movement to published pose is typically 30-100 ms depending on ROS scheduling.

Q5: [Add your own anticipated question]

A5: [Your answer]

5.2 Autonomous Sorting

Owner: Oliver Lau | **Tech stack:** Python / C++ / ROS 2 Humble / MoveIt 2 / ur_onrobot_control

Package(s): ros2_ws/src/moveit_robot_control/ ros2_ws/src/moveit_robot_control_msgs/

5.2.1 Walkthrough Script (approx. 5 minutes)

Tick each step as you cover it. Sections below are in the same order — scroll down as you present.

Walkthrough Script — follow this with your coach

- Step 1.** Orient your coach: 'Autonomous Sorting receives cube poses from Perception and plans + executes collision-free pick-and-place on the UR3e via MoveIt 2 — no operator input needed.'
- Step 2.** Show Purpose & Design Rationale — explain Cartesian-first / pose-goal fallback and why a state machine was used instead of MoveIt Task Constructor.
- Step 3.** Show How It Works — walk through the state machine: pregrasp -> grasp -> lift -> bin -> release -> return.
- Step 4.** Open pick_place_sequencer.py and coordinate_listener.py — point to the state machine transitions and MoveIt planning calls.
- Step 5.** Show Inputs/Outputs — emphasise /holoassist/pick_cube_to_bin as the entry point.
- Step 6.** Demonstrate: ros2 service call /holoassist/pick_cube_to_bin ... '{cube_name: april_cube_1, bin_id: bin_1}'
- Step 7.** Open Parameter Reference — explain hover_offset and planning_time as most commonly tuned.
- Step 8.** Walk through one Troubleshooting row with reasoning.
- Step 9.** Close with Limitations — sequential only, no regrasp, conservative collision avoidance.
- Step 10.** Invite questions.

5.2.2 Purpose & Design Rationale

Purpose:

Receives detected cube poses from Perception and plans and executes collision-free pick-and-place motions using MoveIt 2, the UR3e arm, and the OnRobot RG2 gripper. Operates fully autonomously without operator input, triggered by a single service call naming the cube and destination bin.

Role in the integrated system:

Subscribes to cube poses from Perception via the pose-handoff adapter (which transforms poses from workspace_frame to base_link). Publishes joint trajectories to the ur_onrobot_control driver and gripper commands to /finger_width_controller/commands. Maintains MoveIt planning scene collision objects. Reports progress on /moveit_robot_control/state.

Key design decisions and why:

- Cartesian path first, OMPL pose-goal fallback — straight-line Cartesian approach is predictable for the operator but can fail near singularities; the OMPL fallback finds any collision-free path, ensuring robustness.
- State machine in `pick_place_sequencer.py` rather than MoveIt Task Constructor — simpler to debug and modify without recompiling; each state (pregrasp, grasp, lift, place, release, retreat) is explicit and individually testable.
- `workspace_scene_manager.py` adds collision geometry (trolley mesh, bin walls, workspace bounds) to MoveIt — prevents the planner routing through physical obstacles even if the operator does not see them.
- Service-based entry point rather than topic — a service blocks until completion and returns success/failure, giving the caller explicit feedback; a topic would fire-and-forget.

5.2.3 How It Works — Internal Logic

Point to the implementing code for each step during the walkthrough.

Processing pipeline:

1. Caller triggers `/holoassist/pick_cube_to_bin` with `cube_name` and `bin_id`.
2. `coordinate_listener.py` looks up the latest pose for `cube_name` on `/moveit_robot_control/target_pose` (in `base_link`, from the adapter).
3. State machine: PREGRASP — plans Cartesian path to `hover_offset` above the cube. Executes.
4. GRASP — plans Cartesian descent to grasp pose. Opens RG2 before descent; closes at the bottom.
5. LIFT — Cartesian retreat to hover height. Attaches cube as collision child of gripper in the planning scene.
6. MOVE_TO_BIN — joint-space plan to above the target bin (from `bin_poses.json`). Cartesian descent. Opens RG2.
7. RETREAT — Cartesian path to safe home. Reports COMPLETE or FAILED on `/moveit_robot_control/state`.
8. `workspace_scene_manager` updates collision objects throughout (removes cube from scene on pick, adds to bin on place).

TIP Open the source file and point to the lines implementing each step above.

Data flow:

[Insert: `/holoassist/pick_cube_to_bin` -> `coordinate_listener.py` gets pose -> `pick_place_sequencer.py` state machine -> MoveIt 2 Cartesian/joint plan -> `ur_onrobot_control (/forward_velocity_controller)`; gripper: `/finger_width_controller/commands`; status: `/moveit_robot_control/state`]

5.2.4 Key Files & Code Structure

File / Node	Path	Role	Key Lines to Show
<code>coordinate_listener.py</code>	<code>ros2_ws/src/moveit_robot_control/moveit_robot_control/</code>	Subscribes to <code>target_pose</code> ; calls MoveIt	[Fill in key line ranges]

File / Node	Path	Role	Key Lines to Show
		Cartesian first, pose fallback	
pick_place_sequencer.py	ros2_ws/src/moveit_robot_control/moveit_robot_control/	State machine: pregrasp->grasp->lift->bin->release->return	[Fill in key line ranges]
workspace_scene_manager.py	ros2_ws/src/moveit_robot_control/moveit_robot_control/	Manages MoveIt collision objects (trolley, bins, workspace bounds)	[Fill in key line ranges]
full_holoassist_hardware.launch.py	ros2_ws/src/moveit_robot_control/launch/	Full hardware stack: UR driver + MoveIt + perception + sequencer	All — open whole file
full_holoassist_moveit_sim.launch.py	ros2_ws/src/moveit_robot_control/launch/	Full simulation: fake hardware + MoveIt + perception sim + sequencer	All — open whole file
config/bin_poses.json	ros2_ws/src/moveit_robot_control/config/	Target gripper poses for each bin (world coordinates)	All — open whole file
config/full_holoassist_hw.yaml	ros2_ws/src/moveit_robot_control/config/	Hardware-mode params (workspace TF source, adapter offsets)	All — open whole file

5.2.5 Inputs & Outputs

Inputs:

Input	Source	Data Type / Format
Pick command	Service caller	holo_assist_depth_tracker_sim_interfaces/srv/PickCubeToBin
Cube pose (base_link)	Perception adapter (/moveit_robot_control/target_pose)	geometry_msgs/PoseStamped
Robot joint states	/joint_states from ur_onrobot_control	sensor_msgs/JointState
TF: workspace_frame -> base_link	easy_handeye2 calibration publisher	tf2_msgs/TFMessage

Outputs:

Output	Destination	Data Type / Format	Notes
/moveit_robot_control/state	XR Scene + dashboard	std_msgs/String	QUEUED / PLANNING / PLANNED / EXECUTING / COMPLETE / FAILED
/moveit_robot_control/status	Diagnostics	std_msgs/String	Human- readable current action
/moveit_robot_control/debug	Diagnostics	std_msgs/String	JSON plan details (step, waypoints, timing)
/finger_width_controller/commands	OnRobot RG2 driver	trajectory_msgs/JointTrajectory	Gripper open (0.085 m) / close (0.0 m)

5.2.6 ROS Interfaces

Published topics:

Topic	Msg Type	Rate	Description
/moveit_robot_control/state	std_msgs/String	On change	State machine lifecycle per pick
/moveit_robot_control/status	std_msgs/String	On change	Human-readable current action
/moveit_robot_control/debug	std_msgs/String	On change	JSON plan diagnostics

Subscribed topics:

Topic	Msg Type	Description / How Used
/moveit_robot_control/target_pose	geometry_msgs/PoseStamped	Goal pose in base_link — from the perception-to-motion adapter
/joint_states	sensor_msgs/JointState	Current robot state required for all MoveIt planning

Services / Actions:

Name	Type	Request -> Response	Description
/holoassist/pick_cube_to_bin	PickCubeToBin service	cube_name + bin_id -> success: bool	Blocks until full pick-and-place completes or fails
/sort/cancel	std_srvs/Trigger	[empty] -> success: bool	Aborts in-progress pick and stops robot safely

5.2.7 Running & Testing Independently

Prerequisites:

- UR3e on, remote control mode, External Control program running (host IP 192.168.0.100).
- Full ROS stack: `./launch.sh --robot-ip 192.168.0.194` (or `./launch.sh` for simulation).
- Perception subsystem running and publishing cube poses.

Launch commands:

```
# Simulation
source /opt/ros/humble/setup.bash && source ros2_ws/install/setup.bash
ros2 launch moveit_robot_control full_holoassist_moveit_sim.launch.py
# Hardware
ros2 launch moveit_robot_control full_holoassist_hardware.launch.py
robot_ip:=192.168.0.194
```

Test input:

```
ros2 service call /holoassist/pick_cube_to_bin \
  holo_assist_depth_tracker_sim_interfaces/srv/PickCubeToBin \
```

```
"{cube_name: 'april_cube_1', bin_id: 'bin_1'}"
```

Verify output:

```
ros2 topic echo /moveit_robot_control/state # watch: QUEUED->PLANNING->PLANNED->EXECUTING->COMPLETE
```

EXPECTED State topic cycles QUEUED -> PLANNING -> PLANNED -> EXECUTING -> COMPLETE. On hardware the UR3e physically picks cube 1 and places it in bin_1. In simulation the robot model in RViz follows the planned trajectory.

[Insert screenshot of expected terminal output / headset view here]

5.2.8 Parameter Reference

Coaches often ask 'what happens if I change X?'. The two right columns are the HD differentiator.

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
hover_offset	config/bin_poses.json (per cube)	~0.10 m	0.05-0.20 m	More clearance before descent — safer but slower cycle	Too small -> arm clips object before grasp; too large -> wasted travel time
planning_time	config/full_holoassist_hw.yaml	5.0 s	1.0-30.0 s	More OMPL retries before failure	Too small -> spurious planning failures; too large -> long wait before reporting FAILED
max_velocity_scaling	full_holoassist_hardware.launch.py	0.3	0.05-1.0	Faster motions, shorter cycle time	Too high -> protective stop trigger; too low -> tediously slow

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
max_acceleration_scaling	full_holoassist_hardware.launch.py	0.3	0.05-1.0	Snappier start/stop	Too high -> safety stop; too low -> sluggish
gripper_close_width	pick_place_sequencer.py	0.0 m (fully closed)	0.0-0.085 m	Wider -> softer grip for delicate objects	Too narrow -> motor stall; too wide -> drops object mid-transfer
[Add bin_poses.json entries...]	config/bin_poses.json				

TIP Prepare to demonstrate one live parameter change — edit a value, show the behaviour difference.

5.2.9 Live Configuration Changes

Scenario — "Add a new drop-off bin":

1. Open config/bin_poses.json.
2. Add entry: 'bin_3': {'position': [x, y, z], 'orientation': [qx, qy, qz, qw]} in base_link frame.
3. Reasoning: bin poses loaded at node startup by name string; no code change required.
4. Re-launch and call: ros2 service call /holoassist/pick_cube_to_bin ... '{cube_name: april_cube_1, bin_id: bin_3}'

Scenario — "Slow down for a live demo":

1. Set max_velocity_scaling and max_acceleration_scaling from 0.3 to 0.15 in the launch file.
2. Reasoning: MoveIt scales the TOTG time-parameterisation uniformly — same path at half speed.
3. Re-launch: ros2 launch moveit_robot_control full_holoassist_hardware.launch.py robot_ip:=192.168.0.194

Scenario — "Increase planning robustness in a cluttered scene":

1. Raise planning_time from 5.0 to 10.0 s in config/full_holoassist_hw.yaml.
2. Reasoning: OMPL is a randomised planner; more time = more samples = higher success probability.
3. Trade-off: longer wait before a FAILED result is returned to the caller.

5.2.10 Troubleshooting (with Reasoning)

The 'Why the Fix Works' column separates HD from Credit.

Symptom	Likely Cause & Why	How to Diagnose	Fix	Why the Fix Works
Service returns success: false immediately	Cube pose not available — Perception not running or cube not detected	ros2 topic echo /moveit_robot_control/target_pose — if no messages, Perception is not publishing	Start the Perception subsystem or sim twin; ensure cube is visible to the camera	The sequencer reads the latest available pose; if none exists it cannot form a valid goal and returns false immediately
State stalls at PLANNING beyond planning_time	MoveIt cannot find a collision-free path — goal in collision or scene too cluttered	Open RViz, inspect PlanningScene for unexpected collision geometry at the goal pose	Remove or reposition the blocking object; call workspace_scene_manager to refresh scene	OMPL samples random configuration if the goal state itself is in collision no plan can succeed regardless of time budget
Robot stops mid-motion (protective stop)	Velocity/acceleration scaling too high for current payload + trajectory	Check teach pendant: 'Protective stop'	Lower max_velocity_scaling to 0.2 and max_acceleration_scaling to 0.2; re-launch	UR safety controller enforces joint torque limits; smoother acceleration keeps forces below the threshold
Gripper closes but cube falls mid-lift	Grasp pose too high — gripper closed in air above cube surface	Watch gripper position in RViz during descent; if it goes fully closed before reaching the object the descent ended early	Reduce hover_offset for that cube; or check if workspace_frame->base_link TF calibration is accurate	Grasp depth object_pose.hover_offset; if Perception reports pose too high the gripper never reaches the cube surface
[Add more as discovered...]				

5.2.11 Limitations & Assumptions

Assumptions:

- UR3e URDF accurately reflects physical configuration including RG2 mass and offset.

- All cubes fit within RG2 jaw range (0-85 mm).
- `workspace_frame` locked and accurate before any sort is triggered — stale `workspace_frame` = wrong grasp targets.
- MoveIt planning scene is current; unexpected physical obstacles not listed are not avoided.

Known limitations:

- Sequential sorting only — one cube at a time; no batch or parallel planning.
- No regrasp on slip — if cube slips mid-transfer the cycle fails and must be re-triggered.
- Collision avoidance is conservative — may reject feasible paths when bins are close to workspace boundaries.
- Simulation perception is deterministic — camera noise and occlusion are not modelled.

Future work:

- MoveIt Task Constructor for multi-object batch planning.
- Force-feedback from RG2 to detect and recover from grasp slips.
- Adaptive hover offset based on cube height estimate from Perception.

5.2.12 Anticipated Coach Questions

Pre-prepare answers so you respond confidently.

Q1: Why Cartesian path first, OMPL pose-goal as fallback?

A1: Cartesian paths are straight-line approaches — predictable, visually clear for the operator, and interpretable in MoveIt. However they can fail if a straight-line requires passing through a singular configuration or a collision. The OMPL fallback finds any collision-free path, ensuring the pick succeeds even when the ideal approach is blocked.

Q2: How does the planning scene stay consistent with reality?

A2: `workspace_scene_manager.py` adds collision geometry for the trolley and bins at startup, removes the cube from the scene when picked, and adds it to the bin when placed. If a new physical obstacle appears it must be explicitly added — there is no real-time obstacle detection.

Q3: What is the Perception-to-Sorting handoff contract?

A3: Perception publishes cube poses in `workspace_frame`. The adapter (`selected_cube_to_moveit_target_node.py`) does a TF lookup (`workspace_frame` -> `base_link`) and republishes on `/moveit_robot_control/target_pose` in `base_link`. The full contract is in `docs/POSE_HANDOFF_CONTRACT.md`.

Q4: What if the pick service is called while the robot is already executing?

A4: The service blocks until the current pick completes or fails, then runs the new pick. To abort an in-progress pick, call `/sort/cancel` first.

Q5: [Add your own anticipated question]

A5: [Your answer]

5.3 Teleoperation & Interaction

Owner: Nicholas Sabatta (Nic) | **Tech stack:** C# (Unity 6.3 LTS) / Python (ROS 2 Humble) / PyQt5 / TypeScript+React

Package(s): Unity/My project/Assets/Scripts/ ros2_ws/src/holoassist_unity_bridge/ dashboard/ holoassist-dashboard/

5.3.1 Subsystem Overview

This section provides a high-level technical summary of the Teleoperation & Interaction subsystem. Detailed specifications follow in subsequent sections.

Subsystem Summary — Teleoperation & Interaction

Step 1. The Teleoperation & Interaction subsystem enables real-time operator control of the UR3e arm via three input modes: RMRC (Resolved Motion Rate Control using Jacobian-based velocity commands), Direct Joint jogging, and Hand Guide tracking. It also provides an E-stop safety dashboard, session logging, and a React-based web dashboard for remote monitoring.

Step 2. Three control modes serve distinct operational needs. RMRC provides intuitive Cartesian control via damped least-squares (DLS) Jacobian inversion, which prevents velocity divergence near singularities. Direct Joint jogging allows recovery from singular configurations. Hand Guide enables trajectory demonstration via controller tracking.

Step 3. Control loop architecture: Meta XR SDK controller input (72 Hz) → active mode handler → Jacobian DLS inverse (for RMRC) → exponential smoothing ($\tau = 0.06$ s) → 50 Hz joint velocity commands published to /forward_velocity_controller/commands via ros_tcp_endpoint.

Step 4. Core implementation files: RobotController.cs (mode dispatch, input mapping, 50 Hz publish loop, output smoothing) and UR3eKinematics.cs (forward kinematics, 6×6 geometric Jacobian computation, DLS pseudoinverse).

Step 5. Controller mapping: right thumbstick XY → Cartesian XY velocity; left thumbstick Y → Cartesian Z velocity; right trigger (analog) → gripper width (0–0.085 m); left Menu button → mode cycle; A/B buttons → joint selection (Direct Joint mode).

Step 6. Key tunable parameters: linearSpeed (default 0.25 m/s, requires field tuning), jointJogSpeed (default 0.5 rad/s), outputSmoothTime (0.06 s exponential low-pass time constant), handGuideLateralGain (2.0), handGuideLateralCapM (0.15 m/s).

Step 7. Known critical issue: ROS-TCP-Connector v0.7.0 race condition causes subscriber registrations to be lost after Unity Play/Stop cycles. Workaround: restart ros_tcp_endpoint before re-entering Play mode.

Step 8. Current limitations: end-to-end WiFi latency of 30–80 ms, MeshCollisionGuard disabled pending calibration, linearSpeed and jointJogSpeed values require tuning on physical hardware.

Step 9. Refer to sections 5.3.10 (Troubleshooting) and 5.3.12 (FAQ) for common issues and solutions.

5.3.2 Purpose & Design Rationale

Purpose:

Enables the operator wearing a Meta Quest 3 to directly control the UR3e arm and RG2 gripper in real time across three input modes: RMRC (Resolved Motion Rate Control), Direct Joint jogging, and Hand Guide tracking. Provides a PyQt5 E-stop dashboard, React web dashboard for remote monitoring, and per-session metrics logging.

Role in the integrated system:

Reads Meta XR SDK controller/hand inputs, computes joint velocity commands, and sends them to the UR3e driver via `ros_tcp_endpoint`. Subscribes to `/joint_states` for digital-twin feedback. The E-stop halts motion via burst-zero publishing and controller deactivation.

Key design decisions and why:

- Three modes (RMRC / Direct Joint / Hand Guide) each is optimal for different tasks: RMRC for precise Cartesian approach, Direct Joint for recovering from singular configurations, Hand Guide for trajectory demonstration.
- Damped least-squares (DLS) Jacobian inversion in RMRC — standard Jacobian inversion diverges near singularities; DLS trades a small precision loss for bounded output.
- `ros_tcp_endpoint` (TCP, port 10000) instead of `rosbridge WebSocket` — lower and more predictable latency; `rosbridge` adds JSON serialisation overhead inappropriate for 50 Hz commands.
- E-stop burst-publishes zeros (10x) before switching controllers — ensures stop reaches the robot even with a dropped packet; continuous zero publishing maintains the stop state.
- Session logging runs as a background coroutine so it does not block the 50 Hz control loop.

5.3.3 How It Works — Internal Logic

The following describes the internal processing pipeline and data flow.

Processing pipeline:

1. Unity polls Meta XR SDK at 72 Hz for controller pose, stick axes, and button state.
2. `ROSAutoConnect.cs` TCP-scans 192.168.0.101-109:10000 to find `ros_tcp_endpoint`; connects automatically.
3. Active mode determines command generation: RMRC Translate (stick XY + left stick Y -> Cartesian velocity -> DLS Jacobian inverse -> 6 joint velocities); RMRC Rotate (sticks -> wrist joint velocities directly); Direct Joint (thumbstick Y -> proportional velocity for selected joint); Hand Guide (right grip held + controller 3D delta -> Jacobian IK, capped at 0.15 m/s).
4. Right trigger (analog) maps 0-100% to `/finger_width_controller/commands` (0 = closed, 0.085 m = open).
5. `outputSmoothTime` (0.06 s exponential low-pass) applied to all velocities before publishing at 50 Hz to `/forward_velocity_controller/commands`.
6. `SessionLogger.cs` records mode switches and durations; publishes JSON on `/session/status` at 2 Hz; saves to file on quit.

NOTE See Section 5.3.4 (Key Files & Code Structure) for file paths and relevant line ranges.

Data flow:

[Insert: Quest controller/hand -> Unity Input System -> mode handler -> UR3eKinematics.cs DLS Jacobian -> /forward_velocity_controller/commands -> ros_tcp_endpoint -> ROS -> UR3e. Feedback: /joint_states -> JointStateSubscriber -> digital twin + HUD. E-stop: dashboard button -> burst zeros -> controller switch]

5.3.4 Key Files & Code Structure

File / Node	Path	Role	Key Lines to Show
RobotController.cs	Unity/My project/Assets/Scripts/	Three control modes, gripper mapping, 50 Hz publish, output smoothing	[Fill in key line ranges]
JointStateSubscriber.cs	Unity/My project/Assets/Scripts/	Subscribes to /joint_states; drives robot joint transforms + caches LastJointAnglesRad[]	[Fill in key line ranges]
UR3eKinematics.cs	Unity/My project/Assets/Scripts/	Forward kinematics, geometric Jacobian, DLS pseudoinverse	[Fill in key line ranges]
MeshCollisionGuard.cs	Unity/My project/Assets/Scripts/	Predictive collision protection (table + self-collision) — disabled pending tuning	[Fill in key line ranges]
SessionLogger.cs	Unity/My project/Assets/Scripts/	Mode duration logging, E-stop count, /session/status at 2 Hz	[Fill in key line ranges]
ROSAutoConnect.cs	Unity/My project/Assets/Scripts/	TCP scan 192.168.0.101-109:10000 for auto-discovery of ros_tcp_endpoint	[Fill in key line ranges]
HeadsetStreamPublisher.cs	Unity/My project/Assets/Scripts/	XR camera view -> JPEG -> /headset/image_compressed at 15 FPS	[Fill in key line ranges]
launch.py	root/	Orchestrates: UR driver, controller switch (8 s wait), ros_tcp_endpoint, beacon	All — open whole file
dashboard/main.py	root/dashboard/	PyQt5 E-stop dashboard: 6 tabs, status bar, E-stop + RESUME controls	[Fill in key line ranges]

5.3.5 Inputs & Outputs

Inputs:

Input	Source	Data Type / Format	Notes
Controller pose + buttons	Meta XR SDK (Quest 3)	Vector3 + Quaternion + analog axes	72 Hz — see controller mapping table below
Hand pose (Hand Guide)	Meta Quest 3 hand tracking	OVRHand bone transforms	Right grip held activates hand tracking
/joint_states	ur_onrobot_control via ros_tcp_endpoint	sensor_msgs/JointState	6 joints + finger_width; used for Jacobian + HUD display

Outputs:

Output	Destination	Data Type / Format	Notes
/forward_velocity_controller/commands	ur_onrobot_control	std_msgs/Float64MultiArray	6 joint velocities (rad/s) at 50 Hz
/finger_width_controller/commands	OnRobot RG2 driver	trajectory_msgs/JointTrajectory	Gripper width 0.0-0.085 m proportional to right trigger
/session/status	Dashboard + logging	std_msgs/String	JSON: mode, per-mode durations, E-stop count at 2 Hz
/headset/image_compressed	Dashboard HEADSET tab	sensor_msgs/CompressedImage	XR camera JPEG at 15 FPS

5.3.6 ROS Interfaces**Published topics:**

Topic	Msg Type	Rate	Description
/forward_velocity_controller/commands	std_msgs/Float64MultiArray	50 Hz	6-DOF joint velocity command
/finger_width_controller/commands	trajectory_msgs/JointTrajectory	50 Hz	Gripper position from right trigger

Topic	Msg Type	Rate	Description
/session/status	std_msgs/String	2 Hz	JSON session metrics
/headset/image_compressed	sensor_msgs/CompressedImage	15 Hz	Operator XR camera view

Subscribed topics:

Topic	Msg Type	Description / How Used
/joint_states	sensor_msgs/JointState	Current robot state — drives digital twin + Jacobian computation

Services / Actions:

Name	Type	Request -> Response	Description
/teleop/set_mode (if implemented)	Custom SetMode srv	{mode: string} -> {success: bool}	Switch input mode programmatically

5.3.7 Running & Testing Independently

Prerequisites:

- ./launch.sh (fake hardware) or ./launch.sh --robot-ip 192.168.0.194 (real robot) — provides ros_tcp_endpoint on port 10000.
- Quest 3 in Developer Mode; connected to robot router (192.168.0.x subnet).
- Unity 6.3 LTS open; ROS IP set: Edit -> Project Settings -> ROS Settings -> IP = laptop Wi-Fi IP (e.g. 192.168.0.102), Port = 10000.

Launch commands:

```
# Terminal 1 - full ROS stack (fake hardware)
./launch.sh
# Terminal 1 - real robot
./launch.sh --robot-ip 192.168.0.194
# Terminal 2 - E-stop dashboard
./dashboard.sh --fullscreen
# Unity: File -> Play (or Build and Run to deploy APK to Quest)
```

Test input:

```
# Simulate a joint velocity command without the headset
ros2 topic pub --rate 50 /forward_velocity_controller/commands \
  std_msgs/Float64MultiArray '{data: [0.0, 0.0, 0.1, 0.0, 0.0, 0.0]}'
```

Verify output:

```
ros2 topic hz /forward_velocity_controller/commands # expect 50 Hz when Unity Play +
controller active
ros2 topic echo /session/status
```


EXPECTED Moving the right stick forward in RMRC Translate mode causes the UR3e end-effector to move in the same direction. /forward_velocity_controller/commands reports 50 Hz. Dashboard joint angles update live.

HOLOASSIST
ROS: OFFLINE
CTRL: INACTIVE
JNT: --- Hz
COL: CLEAR
GRIP: OPEN
STATE: RESUMING

STATUS
HEADSET
CAMERA
STATS
LATENCY
SESSION

JOINT STATES

joint_0: waiting...
joint_1: waiting...
joint_2: waiting...
joint_3: waiting...
joint_4: waiting...
joint_5: waiting...

Gripper [.....] 0%

EVENT LOG

[13:03:03] Switching to MOVEIT mode...
[13:03:05] Switching to TELEOP mode...
[13:03:05] Switching to MOVEIT mode...
[13:03:06] Switching to TELEOP mode...
[13:03:06] Switching to MOVEIT mode...
[13:03:07] EMERGENCY STOP TRIGGERED
[13:03:11] Resuming - reactivating controller...
[13:03:13] MOVEIT switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller', '--deactivate', 'forward_velocity_controller', 'finger_width_controller']' timed out after 10 seconds
[13:03:15] TELEOP switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'forward_velocity_controller', 'finger_width_controller', '--deactivate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller']' timed out after 10 seconds
[13:03:15] MOVEIT switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller', '--deactivate', 'forward_velocity_controller', 'finger_width_controller']' timed out after 10 seconds
[13:03:15] TELEOP switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'forward_velocity_controller', 'finger_width_controller', '--deactivate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller']' timed out after 10 seconds
[13:03:16] MOVEIT switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller', '--deactivate', 'forward_velocity_controller', 'finger_width_controller']' timed out after 10 seconds
[13:03:17] Controller deactivation error: Command '['ros2', 'control', 'switch_controllers', '--deactivate', 'forward_velocity_controller', 'finger_width_controller']' timed out after 10 seconds
[13:03:20] Switching to TELEOP mode...
[13:03:21] Switching to MOVEIT mode...
[13:03:21] Controller activation error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'forward_velocity_controller', 'finger_width_controller', '--deactivate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller']' timed out after 10 seconds
[13:03:25] Resuming - reactivating controller...
[13:03:30] TELEOP switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'forward_velocity_controller', 'finger_width_controller', '--deactivate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller']' timed out after 10 seconds
[13:03:31] MOVEIT switch error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller', '--deactivate', 'forward_velocity_controller', 'finger_width_controller']' timed out after 10 seconds
[13:03:35] Controller activation error: Command '['ros2', 'control', 'switch_controllers', '--activate', 'forward_velocity_controller', 'finger_width_controller', '--deactivate', 'scaled_joint_trajectory_controller', 'finger_width_trajectory_controller']' timed out after 10 seconds
[13:03:43] Resuming - reactivating controller...

TELEOP

MOVEIT

EMERGENCY
STOP

5.3.8 Parameter Reference

The following table documents configurable parameters, their valid ranges, and the impact of incorrect values.

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
linearSpeed	RobotController.cs Inspector	0.25 m/s	0.05-0.5 m/s	Faster Cartesian motion per stick deflection	Too high -> overshoot near workspace edges (currently reported too fast — needs

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
					field tuning on real hardware)
jointJogSpeed	RobotController.cs Inspector	0.5 rad/s	0.1-1.5 rad/s	Faster Direct Joint jogging	Too high -> abrupt motion; too low -> unresponsive (currently too slow for wrist joints)
handGuideLateralGain	RobotController.cs Inspector	2.0	0.5-5.0	Controller position delta -> larger EE offset	Too high -> overshoots; too low -> sluggish hand tracking
handGuideLateralCapM	RobotController.cs Inspector	0.15 m/s	0.05-0.3 m/s	Faster Hand Guide maximum speed	Too high -> operator cannot control; safety risk
outputSmoothTime	RobotController.cs Inspector	0.06 s	0.02-0.2 s	Smoother but slower response to input	Too high -> sluggish feel; too low -> jittery (exposes sensor noise)
[Add more Inspector params...]					

NOTE Parameters marked with Inspector locations can be modified at runtime in the Unity Editor without recompilation.

5.3.9 Live Configuration Changes

Scenario — "Switch to Direct Joint mode to escape a singularity":

1. Press left Menu button to cycle to Direct Joint mode.
2. Use right thumbstick Y to jog the selected joint; press A/B to change selected joint.
3. Reasoning: near singularities RMRC Jacobian inversion becomes ill-conditioned; Direct Joint lets you manually move individual joints to escape the configuration.

Scenario — "Lower speed for a public demo":

1. Inspector: select RobotController GameObject; set linearSpeed from 0.25 to 0.10 m/s.
2. Reasoning: the gain multiplies directly into the Cartesian velocity command; halving it gives half-speed for the same stick input.
3. Re-enter Play mode or redeploy APK.

Scenario — "Emergency stop without the dashboard":

1. Use the physical E-stop on the UR3e teach pendant — halts all motion immediately.

2. To resume from the dashboard: hold yellow RESUME button for 5 seconds -> reactivates velocity controllers.
3. Reasoning: the burst-zero publish ensures the stop command arrives even with one dropped packet; the 1-second cooldown prevents accidental re-trigger.

5.3.10 Troubleshooting

The 'Why the Fix Works' column provides the underlying technical rationale for each resolution.

Symptom	Likely Cause & Why	How to Diagnose	Fix	Why the Fix Works
Unity connects but joint angles stay 0.0	ROS-TCP-Connector v0.7.0 race condition — subscriber registrations lost after Unity Play/Stop without restarting the endpoint	ros2 topic hz /joint_states in terminal — if 125 Hz, ROS is fine; issue is Unity-side	Restart ros_tcp_endpoint (Ctrl+C launch.sh; re-run ./launch.sh); then re-enter Play in Unity	ros_tcp_endpoint v0.7.0 does not re-bind subscriber callbacks across Unity Play/Stop without a full restart
Robot drifts when controller is stationary	Sensor noise above effective deadzone threshold	ros2 topic echo /forward_velocity_controller/commands while holding controller still — non-zero = drift	Add or raise software deadzone in RobotController.cs stick input processing before the smoothing filter	A deadzone clips small-magnitude inputs to zero before the low-pass filter, eliminating noise-driven motion
RMRC produces sudden joint snap near workspace edge	Near-singular Jacobian — DLS damping may need tuning for this configuration	Look for snapping at full reach or wrist-aligned configs; log Jacobian condition number in UR3eKinematics.cs	Increase the DLS λ^2 damping factor in UR3eKinematics.cs	Higher lambda bounds the inverse more aggressively near singularities, sacrificing precision but preventing velocity spikes

Symptom	Likely Cause & Why	How to Diagnose	Fix	Why the Fix Works
Quest cannot auto-connect to ros_tcp_endpoint	Quest and laptop on different subnets OR beacon.py not running	ping laptop IP from Quest terminal app; check ./launch.sh started beacon	Confirm both on robot router (192.168.0.x); ensure scan range covers the laptop's Wi-Fi IP (192.168.0.101-109)	ROSAutoConnect.cs scans only 192.168.0.101-109; if laptop Wi-Fi IP is outside this range the scan misses it

5.3.11 Limitations & Assumptions

Assumptions:

- Operator wears Quest 3 and holds both controllers.
- Robot router provides 192.168.0.x subnet with <50 ms RTT between Quest and laptop.
- UR3e in remote control mode with External Control program running.
- MeshCollisionGuard is disabled — operator must self-monitor proximity to table and self-collision configs.

Known limitations:

- Noticeable latency (~30-80 ms) even on fake hardware — partly unavoidable (WiFi + ROS + Unity 72 Hz).
- linearSpeed (0.25 m/s) too fast and jointJogSpeed (0.5 rad/s) too slow — requires field tuning on real hardware.
- MeshCollisionGuard implemented but disabled — breaks arm control until table height and self-collision margins are calibrated.
- ROS-TCP-Connector v0.7.0 race condition requires restarting ros_tcp_endpoint after Unity Play/Stop.

Future work:

- Haptic feedback on controller when nearing joint limits.
- Tune and re-enable MeshCollisionGuard for real hardware safety.
- Replace ros_tcp_endpoint with zenoh for lower-latency telemetry.

5.3.12 Frequently Asked Questions

Common technical questions and detailed answers regarding the Teleoperation & Interaction subsystem.

Q1: What is RMRC and how does the Jacobian work?

A1: RMRC maps desired Cartesian velocity to joint velocities using the geometric Jacobian: $dq/dt = J^+ * dx/dt$. UR3eKinematics.cs computes the 6x6 Jacobian from current joint angles via FK derivatives, then inverts using damped least-squares $(J^T(JJ^T + \lambda^2 * I)^{-1})$ so the output stays bounded even at singular configurations.

Q2: Why three modes instead of one?

A2: Each is optimal for a different task. RMRC for intuitive Cartesian approach. Direct Joint to escape singular or limit-adjacent configurations where RMRC becomes ill-conditioned. Hand Guide for demonstrating or recording trajectories by physically mirroring controller motion.

Q3: How does the E-stop guarantee the robot stops?

A3: The dashboard burst-publishes [0,0,0,0,0,0] ten times, then switches to continuous zero publishing at 50 Hz, then deactivates the controller via ros2 control switch_controllers. The burst covers dropped packets; the controller switch prevents any other node from commanding motion until RESUME is confirmed for 5 s.

Q4: What is the end-to-end control latency?

A4: Typical: Unity computes at 50 Hz (20 ms) -> ros_tcp_endpoint serialisation (~2 ms) -> WiFi (~10-30 ms) -> ROS callback -> UR driver executes. Total ~30-80 ms. At 0.25 m/s the robot moves 8-20 mm before the operator sees the result.

Q5: [Add your own anticipated question]

A5: [Your answer]

5.4 XR Scene & Visualisation

Group Member: Sebastian Baudille | **Software stack:** C# (Unity 6.3 LTS) / Meta XR All-in-One SDK v85.0.0 / C++

Package(s): Unity/My project/

5.4.1 Purpose and Design Rationale

Purpose:

Provides the operator with a complete visual interface inside the Meta Quest 3 headset. Displays live robot joint angles and end-effector pose (RobotDataPanel), per-bin sorting progress (BinStatusPanel), and toggleable overlays (TF axes, passthrough MR/VR, coaching hints) via a user menu.

Role in the integrated system:

Uses /joint_states via JointStateSubscriber (shared with the Teleoperation subsystem). Detects object bin entry through Unity physics trigger events on BinDetector. Publishes the interactive cube pose to /tf and /unity_markers for RViz. All rendering is done within unity.

Key design decisions and why:

- RobotDataPanel reads JointStateSubscriber.LastJointAnglesRad[] rather than subscribing to /joint_states directly as it avoids a condition where the panel's own subscriber starts during Start() before ROSConnection finishes its TCP handshake, which can drop the first messages sent. Polling the shared cached array is more reliable.
- BinDetector builds a dedicated BoxCollider trigger with 5 container walls at runtime from the FBX mesh bounds. FBX bins import with convex MeshColliders that fill the interior volume, causing Rigidbodies to collide with the top of the bin. A non-convex trigger zone built at Start() gives reliable detection.
- RadialMenu uses plane-raycast selection as adding Colliders to the buttons conflicts with the XR Interaction Toolkit's grab/teleport event system. The right controller ray is projected onto the menu's local XY plane and the closest button within the angular arc is selected.
- World-space (not headset-locked) UI panels stay where the operator placed them, reducing motion sickness and keeping the workspace visible.
- ROSObjectPublisher uses System.DateTimeOffset.UtcNow for message timestamps — Unity simulation time is not synchronised to ROS time; using it causes RViz to reject messages as stale.

5.4.3 How It Works

Processing pipeline:

1. JointStateSubscriber subscribes to /joint_states via ros_tcp_endpoint; caches 6 joint angles in LastJointAnglesRad[] and currentFingerWidth on each message.
2. RobotDataPanel reads LastJointAnglesRad[] and currentFingerWidth every frame in LateUpdate() (after JointStateSubscriber has updated). Renders joint angles, EE pose (from UR3eKinematics.cs FK), current teleop mode, gripper %, and EE lock state as a floating world-space quad.
3. Physical or simulated objects with Rigidbodies enter bin areas. BinDetector.OnTriggerEnter/Exit() fires and calls BinStatusPanel.SetBinStatus(binName, count).
4. BinStatusPanel discovers all BinDetectors via FindObjectsByType<BinDetector>() on Enable, then renders per-bin rows (green = occupied, grey = empty).
5. RadialMenu opens on Y button press; plane-raycast from right controller selects buttons; toggles visibility of all panels and overlays.

6. JointTFVisualizer builds RGB cylinder axis markers at every robot joint and every non-static scene object; updated each frame.
7. ROSObjectPublisher reads the interactive cube transform every 0.1 s, converts Unity to ROS coordinates ($rx=uPos.z$, $ry=-uPos.x$, $rz=uPos.y$), and publishes to /tf (unity_cube frame) and /unity_markers at 10 Hz.
8. React dashboard (holoassist-dashboard) subscribes to the same ROS topics via rosbridge and renders 2D panels for remote monitoring.

System Flowchart:

5.4.4 Key Files & Code Structure

File / Node	Path	Role	Key Lines to Show
RobotDataPanel.cs	Unity/My project/Assets/Scripts/	Floating world-space panel — reads JointStateSubscriber cache in LateUpdate()	[Fill in line ranges — point to LateUpdate() and the LastJointAnglesRad read]
BinStatusPanel.cs	Unity/My project/Assets/Scripts/	Per-bin rows; discovers BinDetectors via FindObjectsByType<BinDetector>()	[Fill in line ranges — point to SetBinStatus() and FindObjectsByType call]
RadialMenu.cs	Unity/My project/Assets/Scripts/	Y button opens 13-button circular menu; plane-raycast angular selection	[Fill in line ranges — point to Plane.Raycast call]
BinDetector.cs	Unity/My project/Assets/Scripts/	Attaches to bin FBX; auto-builds BoxCollider trigger + 5 container walls from mesh bounds at Start()	[Fill in line ranges — point to Start() trigger build and OnTriggerEnter()]
JointTFVisualizer.cs	Unity/My project/Assets/Scripts/	RGB cylinder axes on every robot joint + non-static object; toggleable	[Fill in line ranges]
ROSObjectPublisher.cs	Unity/My project/Assets/Scripts/	Publishes interactive cube to /tf + /unity_markers at 10 Hz; Unity->ROS coord conversion	[Fill in line ranges — point to: $rx=uPos.z$, $ry=-uPos.x$, $rz=uPos.y$]
PassthroughToggle.cs	Unity/My project/Assets/Scripts/	MR <-> VR toggle wrapping OVRManager.isInsightPassthroughEnabled	[Fill in line ranges]
CoachingPanel.cs	Unity/My project/Assets/Scripts/	Lightweight operator instruction overlay; toggleable via RadialMenu	[Fill in line ranges]
Editor/HoloAssistSetup.cs	Unity/My project/Assets/Editor/	Menu: GameObject -> HoloAssist -> Setup All UI Features — auto-creates and wires all components	[Fill in line ranges]
SampleScene.unity	Unity/My project/Assets/Scenes/	Single scene: robot model, bins, environment, all panels pre-wired	Open in Editor — show Hierarchy

File / Node	Path	Role	Key Lines to Show
holoassist-dashboard/src/App.tsx	holoassist-dashboard/src/	React entry point — rosbridge WebSocket subscription, panel layout	[Fill in line ranges]

5.4.5 Inputs & Outputs

Inputs:

Input	Source	Data Type / Format	Notes
/joint_states (via JointStateSubscriber cache)	JointStateSubscriber.LastJointAnglesRad[] (6 floats) + currentFingerWidth	float[6] + float	Read in LateUpdate() every frame — avoids Subscribe race condition
XR controller input	Meta XR SDK (Quest 3)	Y button for RadialMenu; right controller for raycast selection	Polled via Unity Input System
Unity physics trigger events	Scene Rigidbodies entering bin trigger zones	Unity internal event (OnTriggerEnter/Exit)	BinDetector updates BinStatusPanel on Rigidbody entry/exit

Outputs:

Output	Destination	Data Type / Format	Notes
/tf	RViz / consumers	tf2_msgs/TFMessage	unity_cube frame pose at 10 Hz (Unity->ROS coord converted)
/unity_markers	RViz	visualization_msgs/Marker	CUBE marker at interactive cube position; 10 Hz
(All other output internal to Unity)			RobotDataPanel, BinStatusPanel, RadialMenu rendered in headset

5.4.6 ROS Interfaces

Published topics:

Topic	Msg Type	Rate	Description
/tf	tf2_msgs/TFMessage	10 Hz	unity_cube pose in base_link (from ROSObjectPublisher)

Topic	Msg Type	Rate	Description
/unity_markers	visualization_msgs/Marker	10 Hz	CUBE marker for RViz

Subscribed topics:

Topic	Msg Type	Description / How Used
/joint_states	sensor_msgs/JointState	Read via JointStateSubscriber cache — NOT subscribed to directly by this subsystem (see Design Rationale § 5.4.2)

Services / Actions:

Name	Type	Request -> Response	Description
(None — visualisation only)			This subsystem does not call or advertise ROS services

5.4.7 Running & Testing Independently

Prerequisites:

- ./launch.sh running — provides ros_tcp_endpoint on port 10000 and /joint_states.
- Unity 6.3 LTS with Meta XR All-in-One SDK v85.0.0 and ROS-TCP-Connector (file:// in manifest.json).
- Quest 3 deployment: Developer Mode + adb installed; or use File -> Build and Run.
- Web dashboard (optional): Node.js 18+; cd holoassist-dashboard && npm install && npm run dev

Launch commands:

```
# Step 1 - Start ROS
./launch.sh
# Step 2 - Unity
# File -> Play (Editor test) OR File -> Build Settings -> Android -> Build and Run (Quest deploy)
# Step 3 - Web dashboard (optional remote monitoring)
cd holoassist-dashboard && npm run dev # open http://localhost:5173
```

Test input:

```
# Publish synthetic joint states to test the panel
ros2 topic pub --rate 10 /joint_states sensor_msgs/JointState \
  '{name: [shoulder_pan_joint, shoulder_lift_joint, elbow_joint, wrist_1_joint, wrist_2_joint, wrist_3_joint], position: [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]}'
# Confirm ROSObjectPublisher is live
ros2 topic echo /tf | grep unity_cube
```

Verify output:

```
# In Unity Play: RobotDataPanel shows the published joint angles
```

```
# Drop a Unity Rigidbody sphere into a bin -> BinStatusPanel row turns green
# Press Y controller button -> RadialMenu opens above left controller
```

EXPECTED Two floating panels appear in the headset: joint data left, bin status right. Pressing Y opens the 13-button radial menu. Dropping a Rigidbody into a bin turns the corresponding row green. ros2 topic echo /tf shows continuous unity_cube output at 10 Hz.

[Insert screenshot of expected terminal output / headset view here]

5.4.8 Parameter Reference

Coaches often ask 'what happens if I change X?'. The two right columns are the HD differentiator.

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
distanceFromCamera	RobotDataPanel Inspector	1.8 m	0.5-3.0 m	Panel further from operator (easier to read at distance)	Too far -> detail hard to read; too close -> blocks workspace view
panelWidth	RobotDataPanel Inspector	0.8 m	0.3-1.5 m	Larger panel, more text visible	Too large -> obstructs workspace; too small -> text unreadable
BinStatusPanel.offset	BinStatusPanel Inspector	(0.45, 0, 0) m	Adjust X to taste	Moves panel left/right relative to left panel	Overlap with RobotDataPanel if X too small
triggerSizeMultiplier	BinDetector Inspector	1.2	0.8-2.0	Larger trigger zone — objects detected further from bin edge	Too large -> false positives from adjacent objects; too small -> objects land in bin but not detected
disableBlockingMeshColliders	BinDetector Inspector	true	true / false	Must be true — disables convex FBX colliders blocking bin interior	If false, Rigidbodies bounce off the convex lid and never enter the bin

Parameter	Location	Default	Valid Range	Effect of Increasing	What Breaks if Wrong
JointTFVisualizer.axisLength	JointTFVisualizer Inspector	0.08 m (joints) / 0.12 m (env)	0.02-0.2 m	Longer RGB axes	Too long -> clutters view; too short -> hard to see orientation
RadialMenu.menuRadius	RadialMenu Inspector	0.1 m	0.05-0.2 m	Larger menu circle, buttons further apart	Too large -> hard to select individual buttons; too small -> accidental selection

5.4.9 Live Configuration Changes

Scenario — "Move the data panel to the right side":

1. Hierarchy: select RobotDataPanel. Inspector: set offset.x from -0.45 to +0.45.
2. Click ForceRefresh() (or exit and re-enter Play mode).
3. Reasoning: RobotDataPanel rebuilds its world-space position from the offset field on OnEnable; changing the Inspector value and refreshing repositions it instantly without code changes.

Scenario — "Add a new bin to BinStatusPanel":

1. Drag a bin FBX into SampleScene.unity.
2. Attach BinDetector.cs component; set binName (e.g. 'bin_3'); check disableBlockingMeshColliders = true.
3. Press Play — BinDetector.Start() auto-builds its trigger zone; BinStatusPanel discovers it via FindObjectsByType<BinDetector>() and adds a new row.
4. Reasoning: discovery is dynamic at runtime — no code change needed to add bins.

Scenario — "Verify coordinate conversion without running the robot":

1. Place the interactive cube at Unity position (0.3, 0.0, 0.5) relative to the robot base origin in the scene.
2. In RViz with base_link as Fixed Frame: the /tf unity_cube frame should appear at ROS position (0.5, -0.3, 0.0).
3. Mapping: ROS X = Unity Z (forward); ROS Y = -Unity X (ROS Y-left vs Unity X-right); ROS Z = Unity Y (both up). If these match, the conversion in ROSObjectPublisher is correct.

Scenario — "Toggle TF axis overlay for debugging":

1. Press Y to open the RadialMenu.
2. Select 'TF Axes' button (page 1).
3. RGB cylinder axes appear at every robot joint and non-static scene object.
4. Reasoning: JointTFVisualizer.Toggle() is called by the RadialMenu; it walks the robot URDF hierarchy and instantiates cylinder primitives — no additional configuration needed.

5.4.10 Troubleshooting

Symptom	Likely Cause & Why	How to Diagnose	Fix	Why th
Panels / objects render pink or magenta on Quest	Universal Render Pipeline/Unlit is excluded from the Android build	Pink materials visible on device only and not in Editor	Project Settings/ Graphics /Always Included Shaders/ add Universal Render Pipeline/Unlit	Unity s referen graph, shader exclude
Joint angles show 0.0 on all joints in the panel	ROS-TCP-Connector race condition (ros_tcp_endpoint not connected when panel first reads cache) OR subscriber not yet registered	ros2 topic hz /joint_states — if 125 Hz, ROS is fine; issue is Unity side	Restart ros_tcp_endpoint (./launch.sh) then re-enter Play in Unity; wait ~2 s for TCP handshake	RobotD the sha JointSt receive not con holds z sympto v0.7.0 l
Rigidbody lands in bin but BinStatusPanel row stays grey	disableBlockingMeshColliders = false — convex FBX MeshCollider blocks Rigidbody entry	In Editor Play mode, select the bin and check Hierarchy for a 'TriggerZone' child object; if absent BinDetector.Start() did not build one	Ensure BinDetector.cs is attached and disableBlockingMeshColliders is checked (true); re-enter Play	With disable false, th fills the Rigidbody 'lid' and OnTrig
Text invisible on Quest (blank quads)	TextMeshPro font not bundled in the Android build	Visible in Editor but blank on device	Assign an explicit TMP_FontAsset to every TextMeshPro component; or place the font under a Resources/ folder	Android unrefere fonts d built-in automa Desktop
ROSOBJECTPublisher shows stale cube in RViz	Clock drift — ROStPublisher was using Unity simulation time (Time.time) instead of wall clock	ros2 topic echo /tf -- check header.stamp; compare to date +%s	Confirm ROStPublisher uses System.DateTimeOffset.UtcNow, NOT Time.time	RViz v stamps clock; U starts a wall clo causing messa

5.4.11 Limitations and Assumptions

Assumptions:

- JointStateSubscriber is present in the same Unity scene and subscribes before RobotDataPanel's LateUpdate runs.
- Physical bins are Unity GameObjects with BinDetector the real passthrough is not used yet.
- ros_tcp_endpoint running and reachable at the configured ROS IP before Unity Play is entered.
- ROStPublisher interactive cube is a placeholder. This will be replaced by the Perception subscription once integration is complete.

Known limitations:

- Bin detection requires Unity physics Rigidbodies.
- ROSObjectPublisher broadcasts a manually-positioned cube and is not yet linked to real Perception output.
- Some environment mesh materials render grey.

Future work:

- Replace ROSObjectPublisher with a subscriber to /holoassist/perception/april_cube_N_pose. This will overlay virtual indicators at real detected cube positions in the workspace.
- Improve the visual fidelity of the simulated environment in Unity.

6. System-Level Troubleshooting

6.1 Common System-Level Issues

6.2 Frequently Asked Questions

Q:
A: